

CLOUD INFRASTRUCTURE SIMULATION

# Security Report

Secure, Reliable & Scalable Infrastructure for an AI Healthcare Platform

DEVSECOPS • CLOUD ENGINEERING • RELIABILITY • OPERATIONS

Prepared for project submission

N210163 | Naga Mahesh Kona  
Rajiv Gandhi University of Knowledge Technologies, Nuzvid  
Phone: 9133889049 | Email: n210163@rguktn.ac.in

Version 1.0 | September 2026

# Overview

This report documents the security posture of the local cloud infrastructure simulation and the checks used to validate the deployment flow.

## Security controls implemented

- Public API only on the required ingress port, while internal services remain private to the Docker network.
- Database and queue services are isolated from the public internet.
- Secrets are injected through environment variables rather than hard-coded into source files.
- A validation pipeline checks for obvious secret-like values before build and deploy stages.
- Health endpoints and container service checks provide deployment gating.
- The repository ignores `.env` files and provides `.env.example` as the safe local template.
- Internal services bind to localhost rather than all host interfaces.
- Custom service containers run as a non-root user.
- Docker Scout is integrated into the pipeline for critical/high fixed CVE detection when authenticated.

## Validation checks in scope

- Secret scanning for risky hard-coded values.
- Compose configuration validation.
- Container build verification.
- Runtime health verification before release completion.
- Simulated deployment gating and rollback readiness.
- Container image vulnerability scanning with Docker Scout.

## Findings

### 1. Missing local environment defaults caused a startup failure

During initial startup, the Compose stack referenced `DB_NAME`, `DB_USER`, and `DB_PASSWORD` without default values. Because no local `.env` file was present, those values resolved to empty strings. The result was a PostgreSQL authentication error and a restart loop in the API container.

### 2. No hard-coded production secrets were committed

The repository keeps environment values in `.env.example` and uses sample placeholders such as `change-me-dev` for local simulation. This is acceptable for a local coursework environment because it avoids committing real secrets while still allowing the stack to boot.

### 3. Public exposure is kept narrow

The public API is the controlled entry point. Internal service ports, database, and queue bind only to localhost for local inspection and remain unavailable to external network clients.

### 4. Deliberately unsafe release is blocked

Running `scripts/deploy_bad_release.ps1` creates `.env.local` with a suspicious database password. The pipeline detects that file before build or deployment and exits at the security stage. `scripts/rollback.ps1`

removes the unsafe override and restores the last healthy local release.

## 5. Image scanning prerequisite

Docker Scout is available in Docker Desktop but requires `docker login` before vulnerability data can be retrieved. The pipeline reports this prerequisite clearly; once authenticated, critical/high fixed vulnerabilities return exit code 2 and block deployment.

## Remediation guidance

- Keep using environment variables and `.env.example` for local configuration values.
- Use default fallback values only for development simulation, not for production.
- Replace placeholder values with a real secret manager in a real deployment environment.
- Require health verification before any release is marked successful.
- Continue to scan for secret-like patterns in IaC and pipeline definitions.
- Authenticate Docker Scout in CI and retain its SARIF or Markdown output as a release artifact.